StreamSketch

Real-Time Fraud Filtering on Payment Data Streams using Bloom Filter, Flajolet-Martin and DGIM. BDA Mini Project Module 4 - Viva Q&A

Dataset: data/paysim_sample.csv (100k rows, 31646 senders, 510 frauds). Demo: 20k rows, FM 15079 exact vs 16384 est (8.65 pct), DGIM 5 vs 4, Bloom TP 12 FP 0. Files: app.py, src/stream.py, src/bloom.py, src/fm.py, src/dgim.py, src/evaluate.py.

A. Project overview

Q: 1. What is your project in one line?

A: StreamSketch: one-pass, low-memory fraud analytics on a payment stream using Bloom Filter, Flajolet-Martin and DGIM. No Hadoop, pure Python plus Streamlit.

Q: 2. What genuine problem does it solve?

A: Payments arrive forever at high rate. With finite memory you must answer live: (a) is this sender a known fraud, (b) how many distinct senders today, (c) how many frauds in the last N payments. Exact needs O(n) memory; sketches do it in O(1).

Q: 3. Which syllabus modules does it cover?

A: Module 4 fully: 4.1 stream model and replay in src/stream.py, 4.3 Bloom filtering in src/bloom.py, 4.4 count-distinct with Flajolet-Martin and combining estimates in src/fm.py, 4.5 counting ones in a window with DGIM in src/dgim.py.

Q: 4. Why no Hadoop?

A: Hadoop/MapReduce is batch-oriented (Modules 1-2). Streaming needs per-event O(1) answers with one look. Sketches are the streaming alternative, so no cluster is needed.

Q: 5. Project structure?

A: app.py (Streamlit dashboard), src/stream.py (generator), src/bloom.py, src/fm.py, src/dgim.py, src/evaluate.py (exact-vs-approx scoring), scripts/download_paysim.py, scripts/make_synthetic.py, scripts/run_evaluation.py, data/paysim_sample.csv, results/metrics.csv.

B. Dataset

Q: 6. Which dataset, and is it one user or all users?

A: PaySim-like mobile-money transactions, same schema as real PaySim. It is ALL users multiplexed: 100,000 rows with 31,646 distinct senders (nameOrig), 510 frauds, 5 types. Each row is one payment event. File: data/paysim_sample.csv.

Q: 7. What do nameOrig, nameDest, isFraud mean?

A: nameOrig is sender account (C...), nameDest is recipient/merchant (M...), isFraud is 1 for fraud else 0. Type is PAYMENT, TRANSFER, CASH_OUT, CASH_IN or DEBIT.

Q: 8. Real vs synthetic data?

A: Full real PaySim (~6M rows, PS_20174392719_1491204439457_log.csv) is on Kaggle under paysim1 and needs login. The app auto-generates a 100k-row PaySim-like CSV with identical columns so the demo works offline. Drop the real file as data/paysim_full.csv and the app picks it up automatically.

Q: 9. How is the stream simulated?

A: src/stream.py reads the CSV row-by-row (chunksize 50000) and yields one dict at a time, optionally with delay. That is the Data-Stream-Management System from 4.1. No rescan, one look per event.

C. Stream model (4.1)

Q: 10. What are stream sources, queries, issues here?

A: Source: payment events. Queries: membership (is fraud?), distinct count, window count. Issues: infinite length, one-pass constraint, finite memory, concept drift, need for approximate answers.

Q: 11. How do you avoid train-test leakage?

A: First half of rows builds the known-fraud blacklist; Bloom stats (TP/FP) are counted only on the second-half unseen rows. FM and DGIM run on the full stream. Caption on the dashboard states this.

D. Bloom Filter (4.3)

Q: 12. How does your Bloom Filter work?

A: Bit array of m bits (default 20000, file src/bloom.py). Add C123: MD5 hash it k=3 times with seeds, set 3 positions to 1. Check C777: hash 3 times; if any position is 0, surely safe; if all 1, maybe fraud, send for DB recheck.

Q: 13. What are m, k, n?

A: m is bits, k is hash functions, n is inserted IDs. Memory is O(m), independent of stream length.

Q: 14. Formula and meaning?

A: FP = (1 - e^(-kn/m))^k. More IDs (n) or more hashes (k) raise fill and FP; larger m lowers FP. Dashboard shows observed FP next to theory; both are 0.0000 in the 20k demo because the blacklist is small.

Q: 15. What are TP, FP, FN here? Why is FP acceptable?

A: TP: flagged and truly fraud. FP: flagged but actually safe. FN never happens in Bloom: a real fraud is never missed. FP only costs one extra DB lookup, so it is acceptable.

Q: 16. Demo numbers?

A: 20k run: known ~39 fraud IDs, TP 12, FP 0 on unseen half. Memory 2500 B. At demo scale exact set (39x12 B = 468 B) looks smaller; at 1M fraud IDs exact is ~12 MB while Bloom stays 2.5 KB. Quote the bank-scale column.

E. Flajolet-Martin (4.4)

Q: 17. How does FM estimate distinct users?

A: Hash each sender to 32 bits, count trailing zeros. Rare patterns imply many users. Track max zeros R per hash; estimate is 2^R. Example: max 19 zeros implies about 2^19 = 524k uniques.

Q: 18. Why 24 hashes and median?

A: Single FM has high variance. Syllabus calls for combining estimates. We keep 24 independent hashes (seeds 0..23) and take the median of the 24 estimates. That is src/fm.py estimate().

Q: 19. Memory and accuracy?

A: 24 ints = 96 bytes vs 15079 IDs x 12 B = ~177 KB exact in the 20k run. Demo: exact 15079, estimate 16384, error 8.65%. At 10M users exact is ~120 MB, FM stays 96 B.

Q: 20. What if the reviewer asks why estimate is a power of two?

A: Each 2^R is a power of two; with 24 hashes the median of an even count can average two middle values (e.g. 40960), so non-powers can appear. It is still correct FM behavior.

F. DGIM (4.5)

Q: 21. What problem does DGIM solve?

A: Counting ones in a sliding window: how many frauds in the last N payments (default N=1000). Exact needs the full window; DGIM needs O(log N) buckets.

Q: 22. How do buckets work?

A: Each fraud bit creates a size-1 bucket with timestamp. At most 2 buckets per size (1,2,4,8...); if 3 appear the oldest two merge into one double-size bucket stamped with the newer time. Buckets older than time-N expire. See src/dgim.py _compress and _evict.

Q: 23. Query formula?

A: count = sum(bucket sizes) - oldest_size/2. Subtracting half the oldest bucket corrects for the partially-expired bucket. Demo: exact 5, estimate 4, using 3 buckets vs 1000 slots.

Q: 24. What is a decaying window?

A: Old events matter less. DGIM decays by expiring old buckets; merged-bucket timestamps keep the window sliding without storing every bit.

G. Evaluation and UI

Q: 25. What do the 4 live cards mean?

A: Processed i/N: rows streamed, proves one pass. Bloom hits TP/FP: fraud catches vs false alarms. Distinct FM/exact: sketch vs set. Frauds last N DGIM/exact: window sketch vs deque sum.

Q: 26. What do the 3 final cards mean?

A: Bloom FP observed vs theory (0.0000/0.0000). FM error 8.65% with 96 B vs 177 KB. DGIM 3 buckets vs 1000 slots.

Q: 27. What does the memory table prove, and what are the None cells?

A: Each row has one relevant scale column: Bloom at 1M fraud IDs, FM at 10M users, DGIM at N=100k. None means that column does not apply to that row. Read only the filled cell per row.

Q: 28. What do sidebar controls do?

A: Rows: stream length (20k best for viva). m/k: Bloom memory vs FP tradeoff. N: window length. Refresh: UI redraw batch only, does not change results. Run stream: rebuilds blacklist and replays once.

Q: 29. CLI numbers for the report?

A: Run python scripts/run_evaluation.py --n 20000. It writes results/metrics.csv: n, bloom_fp_rate, theory, fm_exact, fm_estimate, fm_error_pct, dgim_exact, dgim_estimate.

H. Limits and extensions

Q: 30. Limitations?

A: Approximate answers with bounded error; Bloom needs the fraud list seeded first and cannot delete; FM variance needs many hashes; DGIM assumes binary stream; synthetic data is simpler than real drift.

Q: 31. How would you extend it?

A: Count-Min Sketch for heavy hitters, HyperLogLog for tighter distinct counts, decaying counts with timestamps, Kafka ingestion instead of CSV replay, MongoDB blacklist to also cover Module 3, precision-recall over thresholds.

Q: 32. Why is exact kept in code if production would drop it?

A: Exact set and deque run alongside only to measure error percent for the report. In production they are deleted and only the sketches remain.

Q: 33. 30-second closing line?

A: Payments arrive faster than we can store, so exact needs O(n). We do one pass with three O(1) sketches: Bloom for membership, FM for distinct count, DGIM for window count, with measured errors 0, 8.6 percent and off-by-one.